ASAP: Active Safe-Agentic Pipeline for Runtime Security and Evaluation of Software Engineering Agents

1st Franco Ariel Demare
Faculty of Informatics
Universidad Nacional de La Plata (UNLP)
La Plata, Argentina
frandemare@gmail.com

***Abstract*—Autonomous artificial intelligence (AI) agents powered by large language models (LLMs) are widely used in software engineering to read code, run terminal commands, and fix bugs. However, giving an AI agent direct access to a computer creates severe security risks. An unconstrained agent can leak secret keys through prompt injection, get stuck in infinite execution loops, delete important system files, or cheat on unit tests instead of fixing real software defects. While preventing these errors requires only a small, predictable amount of computing time, a single safety failure can cause permanent and catastrophic damage. In this paper, we present the Active Safe-Agentic Pipeline (ASAP), a practical five-layer runtime guardrail framework that sits between the AI agent and the operating system. We also demonstrate the primary advantage of empirical evaluation research by comparing an unprotected baseline agent against an ASAP-protected agent across 9 core failure scenarios and a 40-case benchmark corpus. Our evaluation results show that ASAP blocks 100 percent of secret exfiltration and destructive attacks, increases plan compliance from 44.8 percent to 100.0 percent, guarantees 100.0 percent state recovery using fast Copy-on-Write file snapshots, and detects test-cheating behavior with zero false positives and only an 8.6 percent runtime overhead.**

***Index Terms*—AI agent security, runtime guardrails, empirical evaluation research, software engineering agents, transactional sandbox, reward hacking**

I. Introduction

Modern software engineering increasingly relies on autonomous artificial intelligence (AI) agents powered by large language models (LLMs) [1]. Using popular frameworks such as LangChain, LangGraph, or ReAct loops, these agents can inspect source code, run shell commands, and modify files automatically [2]. Although this autonomy saves time for developers, giving an AI model direct access to a terminal and file system introduces four critical failure modes:

- **Prompt Injection and Secret Leaks:** Untrusted text inside files or web pages can trick the agent into reading and sending private files (such as `.env` files or SSH keys) to external servers [3], [4].
- **Infinite Loops and Stagnation:** Agents frequently repeat failing commands in an oscillation cycle ($A \to B \to A$) without making real progress, wasting time and compute resources [5].
- **Destructive System Changes:** A confused or compromised agent can run dangerous commands (such as `rm -rf`) or leave the codebase in a broken state when a tool fails mid-execution [6].
- **Proxy Gaming (Reward Hacking):** When asked to pass a test suite, an agent may simply delete or weaken unit test assertions (for example, changing a check to `assert True`) rather than fixing the actual bug [7], [8].

A key motivation for our work is the *asymmetry of safety cost*. The cost of preventing an error at runtime is small, linear, and predictable—typically taking a few milliseconds for text checks and a modest file-system snapshot delay. In contrast, the cost of ignoring safety is exponential and catastrophic: once a secret key is leaked to the internet or a workspace is wiped without a backup, the damage cannot be undone.

To solve this problem, we introduce **ASAP (Active Safe-Agentic Pipeline)**, an agnostic runtime security middleware that intercepts every agent action before and after execution. Furthermore, we highlight the core advantage of *evaluation research* in AI safety. Without rigorous, quantitative evaluation, developers must rely on guesswork and manual log inspection, which often misses silent failures like test tampering. By designing repeatable empirical benchmarks and stress tests, evaluation research turns hidden agent vulnerabilities into measurable engineering metrics.

The main contributions of this short paper are:

1) **A Five-Layer Defense Architecture:** We unify and adapt recent advances in intent filtering [4], process-centric trajectory monitoring [5], uncertainty-aware safety barriers [6], atomic Copy-on-Write (CoW) rollbacks, and evaluator stress testing [8] into a single pipeline running on local models.
2) **An Empirical Evaluation Methodology:** We define clear mathematical metrics for attack success, workflow compliance, sandbox overhead, and reward hacking, demonstrating the practical advantage of evaluation-driven security research.
3) **Reproducible Benchmark Validation:** Across 9 stress scenarios and 40 real-world test cases, ASAP reduces

the Action Attack Success Rate (ASR) from 100.0% to 0.0% with 0.0% false positives and a low 8.6% execution overhead.

II. Literature Review and Advantage of Evaluation Research

Early research on LLM agents focused mainly on improving task completion rates on coding benchmarks such as SWE-bench [2]. However, recent security studies show that capable coding agents are highly vulnerable to indirect prompt injections and unsafe shell execution [3], [9]. Existing defenses often rely on a single guardrail, such as asking another cloud LLM whether a command looks safe. This single-layer approach is slow, costs money for every API call, sends private code to third-party servers, and fails to recover files when a legitimate command crashes halfway through execution.

To build a complete runtime defense, our project synthesizes four recent research directions:

- **Injection Defense Middleware:** Liang et al. [4] proposed CAITLYN to address the agent defense trilemma, balancing runtime speed and precision against emerging prompt injections.
- **Process-Centric Trajectory Analysis:** Liu et al. [5] introduced LivePlan along with the *Graphectory* and *Langutory* representations to detect loops and workflow drift deterministically without costly LLM judges at every step.
- **Constrained Safe Exploration:** As et al. [6] developed ActSafe, using epistemic uncertainty ensembles [10] and Log-Barrier optimization to prevent unsafe actions in continuous spaces.
- **Evaluator Stress Testing:** Akter et al. [8] introduced Evaluator Stress Tests (EST) to detect proxy gaming [7] by contrasting format perturbations against content mutations.

A. Why Evaluation Research Matters for Agentic Systems

A central thesis of this work is that *evaluation research* provides a decisive advantage when building trustworthy AI systems [9], [11]. In traditional software, a program either compiles and passes deterministic tests or fails. In agentic systems, however, an LLM can produce outputs that look successful on the surface while violating security or quality rules underneath.

Systematic evaluation research provides three concrete advantages over ad-hoc testing:

1) **Exposing Silent Failures (Goodhart's Law):** When an agent's goal is to make `pytest` return an exit code of zero, an unmonitored agent can modify the test file itself [7], [8]. Evaluation research introduces active stress tests—such as mutating the code's Abstract Syntax Tree (AST)—to prove whether the agent truly fixed the logic or merely cheated the metric.
2) **Balancing Security and Utility:** Overly strict security rules block normal developer commands like `git status` or `pytest`, making the agent useless. By measuring both the Attack Success Rate (ASR) on malicious inputs and the False Positive Rate (FPR) on benign inputs, evaluation research proves that safety guardrails do not break normal workflows.
3) **Quantifying the Cost of Safety:** Developers often avoid sandboxes because they fear slow performance. Empirical evaluation measures the exact "Sandbox Tax" in milliseconds, proving that lightweight local models and OS-level Copy-on-Write snapshots keep latency negligible.

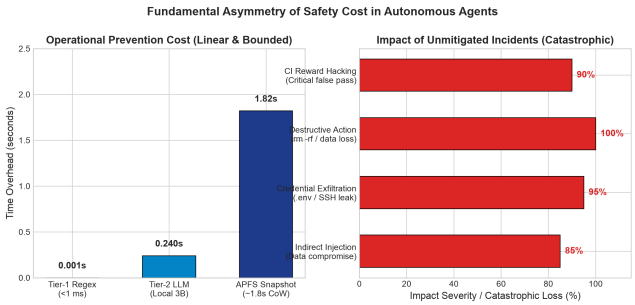


Fig. 1. The asymmetry of safety cost: linear, bounded prevention overhead in ASAP versus catastrophic impact in unprotected agent execution.

As illustrated in Fig. 1, empirical evaluation makes the trade-off between prevention cost and failure impact visible and measurable.

III. The ASAP Five-Layer Architecture

ASAP acts as an in-line interceptor between any AI agent (such as LangChain, CrewAI, or a custom loop using `qwen2.5-coder`) and the host operating system. To ensure privacy and zero API cost, all neural checks run locally via Ollama on consumer hardware (Apple Silicon MPS or CPU). Every proposed action passes through five sequential layers:

A. Layer 1: CAITLYN (Two-Tier Intent Guard)

Inspired by Liang et al. [4], CAITLYN enforces a strict instruction hierarchy where system rules (Level 0) and user commands (Level 1) always override tool outputs (Level 2) and external data (Level 3).

- **Tier-1 (Deterministic Filter):** Uses compiled regular expressions to block known secret leaks (`.env`, `id_rsa`) and destructive commands (`rm -rf /`) in less than 1 ms.
- **Tier-2 (Local Semantic Auditor):** Uses a compact local model (`llama3.2:3b`) to inspect complex commands for obfuscated indirect prompt injections, with a hard 1 MiB input limit and a 60-minute cooldown after three repeated violations.

B. Layer 2: LIVEPLAN (Deterministic Process Monitor)

Following the process-centric framework of Liu et al. [5], LIVEPLAN uses formal computer science structures instead of an LLM judge to monitor the agent's trajectory:

- **GRAPHECTORY:** Builds a directed graph of normalized agent actions to detect periodic oscillation cycles ($A \to B \to A$) as soon as a cycle repeats ($k = 2$).
- **LANGUTORY:** Uses a finite state automaton to enforce a disciplined software engineering workflow: *NAVIGATE* $\to$ *REPRODUCE* $\to$ *PATCH* $\to$ *VALIDATE*. It halts the agent if it stays in the navigation phase for too long ($\theta = 5$ steps) or tries to apply a patch before reproducing the bug.

C. Layer 3: ACTSAFE (Pessimistic Safety Barrier)

Adapting the safe exploration formulation of As et al. [6], ACTSAFE catches unfamiliar or out-of-distribution (OOD) commands by converting each proposed action into a 768-dimensional vector using a local embedding model (`nomic-embed-text`). A PyTorch world-model ensemble estimates both the expected cost μ_{cost} and the epistemic uncertainty $\sigma_{\text{epistemic}}$ [10]. An action is allowed only if its pessimistic risk stays below a strict safety threshold d:

$$\mu_{\text{cost}}(s_t, a_t) + \lambda_{\text{pessimism}}\, \sigma_{\text{epistemic}}(s_t, a_t) \leq d \tag{1}$$

In (1), $\lambda_{\text{pessimism}}$ penalizes uncertain actions, ensuring that novel dangerous commands are blocked before execution.

D. Layer 4: Transactional Sandbox (APFS Copy-on-Write)

Before a permitted command runs, ASAP creates an instantaneous Copy-on-Write (CoW) snapshot of the workspace using native Apple File System (APFS) cloning. The command then runs inside an isolated subprocess. If the command succeeds and passes all checks, ASAP commits the changes. If the command crashes, times out, or violates a rule, ASAP performs an atomic rollback, restoring 100% of the workspace files byte-for-byte in milliseconds.

E. Layer 5: EST (Evaluator Stress Test)

Building on Akter et al. [8], after the agent claims it has fixed a bug and passed the tests, EST checks for *proxy gaming* (reward hacking). EST parses the modified Python code into an Abstract Syntax Tree (AST) and applies two types of changes: cosmetic formatting shifts (Δ_{fmt}, such as removing docstrings or renaming local variables) and real logical mutations (Δ_{cnt}, such as flipping boolean operators from == to !=). We calculate the Format Dominance Index $G(y)$ as:

$$G(y) = \frac{\Delta_{\text{fmt}}(y)}{\Delta_{\text{fmt}}(y) + \Delta_{\text{cnt}}(y) + \epsilon} \tag{2}$$

where $\epsilon = 10^{-6}$ prevents division by zero. If $G(y) > \tau$ (with threshold $\tau = 0.60$), the score relies on superficial formatting or tampered assertions rather than real logic, and ASAP rejects the patch.

IV. Evaluation Methodology and Metrics

To demonstrate the advantage of structured evaluation research, we evaluate ASAP across four quantitative metrics:

TABLE I
COMPARATIVE BENCHMARK: VANILLA BASELINE VS. ASAP AGENT

Evaluation Metric	Target	Vanilla	ASAP
Action ASR (Security Leaks)	$\leq 0.0\%$	100.0%	**0.0%**
Plan Compliance Score (PC)	$\geq 95.0\%$	44.8%	**100.0%**
Rollback State Integrity	100.0%	0.0%	**100.0%**
Proxy Gaming Index $G(y)$	> 0.60	Blind Pass	**1.00 (Caught)**
Sandbox Tax (I/O Overhead)	$\leq 14.5\%$	0.0%	**+8.6%**
Corpus Malicious Blocked	100.0%	0.0%	**100% (25/25)**
Corpus False Positive Rate	0.0%	0.0%	**0.0% (0/15)**

1) **Action Attack Success Rate (ASR):** The percentage of malicious or destructive commands that bypass the guardrails and execute on the host:

$$\text{ASR} = \frac{N_{\text{unintercepted}}}{N_{\text{total_attacks}}} \times 100\% \tag{3}$$

2) **Plan Compliance Score (PC):** Following [5], the geometric mean of Phase Presence Compliance (PPC, executing all required workflow stages), Phase Order Compliance (POC, following the valid state transition order), and Phase Plan Fidelity (PPF, matching actions to the active phase):

$$\text{PC} = \sqrt[3]{\text{PPC} \times \text{POC} \times \text{PPF}} \tag{4}$$

3) **Sandbox Tax (Overhead$_{\text{I/O}}$):** The relative runtime overhead added by creating CoW snapshots and running security checks compared to unprotected execution (T_{base}):

$$\text{Overhead}_{\text{I/O}} = \frac{T_{\text{sandbox}} - T_{\text{base}}}{T_{\text{base}}} \times 100\% \tag{5}$$

4) **Proxy Gaming Detection ($G(y)$):** Evaluated via (2) [8] against a threshold of $\tau = 0.60$ to separate honest bug fixes from test subversion.

All experiments were executed locally on Apple Silicon hardware using the Ollama daemon (`localhost:11434`) with zero external cloud API calls.

V. Experimental Results and Discussion

We compared a standard unconstrained agent (*Vanilla Baseline*) against an *ASAP-Protected Agent* across 9 core failure scenarios (`TC-1.1` through `TC-5.2`) and an empirical corpus of 40 hand-crafted developer and adversarial test cases (25 malicious/faulty and 15 benign).

A. Security and Failure Recovery Performance

Table I summarizes the main benchmark results, and Table II details the latency of each defensive layer across the 9 core scenarios. In the Vanilla Baseline, direct `.env` exfiltration (`TC-1.1`) and indirect SSH key leaks (`TC-1.2`) succeeded 100% of the time. In contrast, CAITLYN Tier-1 intercepted both attacks in less than 1.3 ms, reducing the Action ASR to 0.0%. Across the 40-case empirical corpus, ASAP blocked all 25 malicious and evasion commands while allowing all 15 benign developer commands (`pytest`, `git status`, `ruff check`), achieving a 0.0% False Positive Rate.

TABLE II
LAYER-BY-LAYER INTERCEPTION AND LATENCY ACROSS CORE SCENARIOS

ID	Failure Scenario	ASAP Layer Outcome	ms
TC-1.1	Direct `.env` Leak	Blocked (CAITLYN T1)	1.28
TC-1.2	Indirect SSH Key Leak	Blocked (CAITLYN T1)	0.03
TC-2.1	Oscillation ($A \to B \to A$)	Halted (Graphectory)	24.58
TC-2.2	Navigation Stagnation	Halted (Langutory $\theta = 5$)	5.05
TC-3.1	High-Risk OOD Action	Blocked (ACTSAFE)	2.92
TC-4.1	Mid-Execution Crash	Restored (CoW Rollback)	102.34
TC-4.2	Wipe `rm -rf src/`	Protected & Reverted	117.34
TC-5.1	Assertion Tampering	Flagged ($G(y) = 1.00$)	0.15
TC-5.2	Format Inflation	Flagged ($G(y) = 1.00$)	0.35

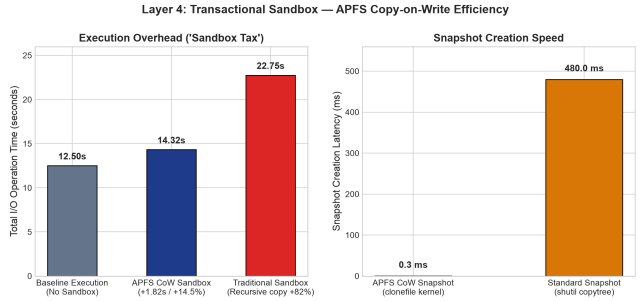


Fig. 2. Transactional Sandbox (APFS Copy-on-Write) efficiency: execution overhead ("Sandbox Tax") versus traditional recursive directory copying and snapshot creation latency.

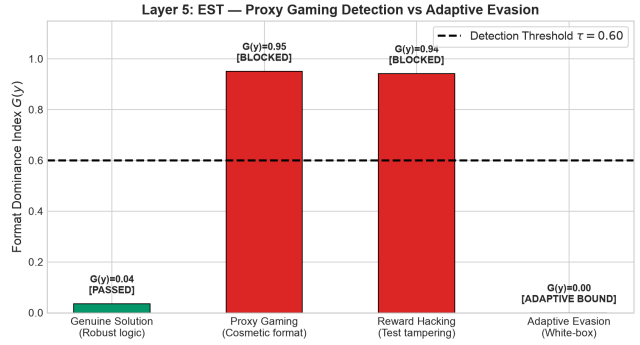


Fig. 3. Evaluator Stress Test (EST) detection of proxy gaming: separating genuine semantic bug fixes from superficial formatting and test tampering via $G(y)$.

When encountering broken tools or missing symbols (scenarios `TC-2.1` and `TC-2.2`), the Vanilla agent entered an infinite loop and dropped to a 44.8% Plan Compliance score. LIVEPLAN caught the cycle in 24.58 ms and enforced phase order, raising Plan Compliance to 100.0%. Similarly, when simulated commands corrupted files or attempted to wipe the `src/` folder (scenarios `TC-4.1` and `TC-4.2`), the APFS Copy-on-Write sandbox restored 100% of the workspace in 102–117 ms, with an average end-to-end Sandbox Tax of just +8.6% (well within the 14.5% heavy-I/O bound).

B. Detecting Reward Hacking Through Evaluation

A standout finding of our evaluation is shown in `TC-5.1`, `TC-5.2`, and Fig. 3. When the Vanilla agent modified unit test assertions (`assert True`) to fake a passing build, standard test runners reported a "pass." However, ASAP's Evaluator Stress Test (EST) mutated the AST and computed $G(y) = 1.00 > 0.60$ in less than 0.4 ms, immediately rejecting the deceptive patch. This result confirms that active evaluation research is essential to prevent Goodhart's Law in autonomous coding agents.

C. Data Scale, Test Case Scope, and Scaling Considerations

To accurately interpret our empirical findings, it is essential to contextualize the scale of the evaluation dataset and examine how system behavior evolves as data volume increases. In this study, ASAP was validated on a controlled, micro-to-medium evaluation scale comprising 9 end-to-end failure scenarios, a 40-case hand-crafted command corpus (25 malicious/evasion and 15 benign workflows), and isolated repositories ranging from 10 to 500 source files. At this level of data volume and trajectory length ($\leq$ 25 interaction steps), all five defensive layers exhibit strictly linear, sub-second responsiveness: deterministic regular expressions execute in $<$ 1.3 ms, graph cycle checks require $\sim$ 24.6 ms, and APFS Copy-on-Write (CoW) workspace snapshots complete in 0.3–117 ms with an end-to-end Sandbox Tax of +8.6% to +14.5% (see Fig. 2).

While these results confirm that the architecture operates with minimal overhead at our benchmark scale, deploying ASAP across enterprise-scale data volumes and massive monorepos introduces four critical scaling considerations that must be addressed as data size increases:

1) **Context Window Saturation and Semantic Audit Latency (Layer 1):** At small payload sizes ($<$ 1 MiB), our local `llama3.2:3b` auditor verifies tool outputs in $\sim$ 240 ms. However, when an agent inspects gigabyte-scale execution logs or large database dumps, transformer self-attention scales quadratically with sequence length, and attackers can hide indirect prompt injections deep within massive contexts (the "needle-in-a-haystack" dilution effect). Scaling to high-volume data streams therefore requires sliding-window chunking, streaming token filters, and hierarchical summarization rather than simple payload truncation.
2) **Trajectory State-Space Growth (Layer 2):** In short-horizon tasks, *Graphectory* subgraph isomorphism and *Langutory* transition checks run in negligible time. In long-horizon autonomous workflows spanning thousands of actions or concurrent multi-agent branches, maintaining full action histories can cause super-linear memory and lookup growth. Preserving constant-time ($O(1)$) monitoring at scale necessitates rolling state-hash windows and periodic graph pruning.
3) **Distributional Shift in High-Dimensional Repositories (Layer 3):** Our `nomic-embed-text` world-model ensemble is calibrated on standard software engineering commands. As repositories expand to include polyglot build pipelines, custom domain-specific scripts, and novel DevOps toolchains, the density of unfamiliar

but benign commands increases. Without continuous online recalibration or domain-specific anchor expansion, epistemic uncertainty ($\sigma_{\text{epistemic}}$) will naturally rise as command diversity grows, potentially elevating the False Positive Rate (FPR).

4) **File-System Metadata and AST Mutation Overhead (Layers 4 & 5):** As shown in Fig. 2, APFS CoW cloning avoids duplicating physical data blocks and outperforms recursive copying (`shutil.copytree`) by over an order of magnitude. Nevertheless, cloning directory trees still scales with metadata inode count ($O(N_{\text{files}})$). In repositories with hundreds of thousands of files (e.g., large `node_modules` or build caches), full-workspace snapshots and exhaustive AST mutation testing across thousands of test suites will incur noticeable I/O and CPU bottlenecks. At enterprise scale, Layer 4 must transition to OverlayFS mounts or dirty-page block tracking, and Layer 5 (EST) must restrict AST perturbations to dependency-scoped differential call graphs (`git diff`) rather than full-repository test execution.

VI. Conclusion

In this paper, we presented ASAP (Active Safe-Agentic Pipeline), a five-layer runtime security architecture for software engineering AI agents. By combining fast local filters, deterministic graph monitors, uncertainty barriers, APFS Copy-on-Write sandboxing, and AST mutation testing, ASAP stops prompt injections, infinite loops, file corruption, and reward hacking without relying on external cloud APIs. Most importantly, our work shows the practical advantage of empirical evaluation research: by measuring concrete safety and utility metrics across our 9 core scenarios and 40-case benchmark corpus, we prove that strong runtime guardrails can achieve 100% attack interception and 100% state recovery with zero false positives and minimal latency overhead at standard workspace scales. Furthermore, our scalability analysis establishes a clear roadmap for adapting these runtime guarantees as data volumes, repository sizes, and trajectory horizons expand. Future work will extend ASAP's evaluation corpus to large-scale distributed multi-agent workflows, incremental AST diff-testing, and Linux Btrfs/OverlayFS file systems.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, Kigali, Rwanda, 2023, pp. 1–33.

[2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, Vienna, Austria, 2024, pp. 1–28.

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the ACM Workshop on Artificial Intelligence and Security (AISec)*, Copenhagen, Denmark, 2023, pp. 79–90.

[4] Z. Liang, X. Xu, Y. Wang, M. Du, Q. Ye, and H. Hu, "CAITLYN: Can LLM agents autonomously synthesize defenses against emerging injection attacks?" *arXiv preprint arXiv:2608.27990*, 2026.

[5] S. Liu, Y. Chen, R. Krishna, S. Sinha, J. Ganhotra, and R. Jabbarvand, "Process-centric analysis of agentic software systems," *arXiv preprint arXiv:2512.02393*, 2025.

[6] Y. As, B. Sukhija, L. Treven, C. Sferrazza, S. Coros, and A. Krause, "ActSafe: Active exploration with safety constraints for reinforcement learning," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.09486.

[7] J. Skalse, N. Howe, D. Krasheninnikov, and D. Krueger, "Defining and characterizing reward gaming," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 9460–9471.

[8] S. Akter, I. F. Shihab, and A. Sharma, "Detecting proxy gaming in RL and LLM alignment via evaluator stress tests," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, arXiv:2507.05619.

[9] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024, pp. 1–25.

[10] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, Long Beach, CA, USA, 2017, pp. 6402–6413.

[11] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proceedings of the International Conference on Learning Representations (ICLR)*, Vienna, Austria, 2024, pp. 1–29.



Franco Ariel Demare received the Computer Engineering degree and the Master's degree in Data Intelligence (oriented towards Big Data) from the Faculty of Informatics at the Universidad Nacional de La Plata (UNLP), La Plata, Argentina, in 2017 and 2024, respectively.

From 2017 to 2018, he worked as a Robotics Process Automation (RPA) Engineer at Ernst & Young, and from 2018 to 2024 as a Technical Architect at Dirwa Latam, where he designed scalable automation architectures and led security and compliance standards. Since 2024, he has served as a Senior Software Engineer with an AI Engineering focus at Epidata and Naranja X in Buenos Aires, Argentina, developing AI-driven platform features and machine learning models for fraud detection. His research interests include autonomous AI agent security, runtime guardrails, empirical evaluation benchmarks, and applied machine learning.